Exercises

System Calls

Complete the below assignments in ls/SystemCalls:
1. Use access system call to check the read & write access for the file (Refer check_access.c)
2. Use access system call to check the executable permission for the file (Modify check_access.c)
3. Use fcntl to place the write lock on the file (Refer fcntl.c & complete the TODOS)
4. Use fcntl to place the read lock on the file and try the following:
  - Try running 2 programs with read lock and observe the behaviour
  - Try running one program with read lock and another write lock and observe the behaviour
   NOTE: Copy fnctl.c as fcntl_rd.c and make the above changes
5. Use fcntl to place the write lock on the file region (copy fcntl.c as fcntl_wrlk.c)
6. Get the status of the fcntl lock using F_GETLK (Enhance fcntl_wrlk.c)
  (i) Acquire the fcntl write lock only if status of lock shows as unlocked, otherwise keep on retrying till the lock is available make sure to use the sleep during the busy waiting.
7. Use gettimeofday, localtime & strftime to display the time. (Refer print_time.c)
8. Use rlimit to limit the processor time for the process (Refer rlimit.c)
9. Use sysinfo to get the information such as uptime, Total RAM, Free RAM and process count (Refer sysinfo.c & complete all the todos)

Processes

Get into the Processes Directory
Complete following:
1. Print process and parent process id (Try pid.c)
2. Use system function to print the ls -l results (Try system.c)
3. Use system function to print the ps -x results
4. Demonstrate the usage of fork (try fork.c)
5. Verify the fork tree (try fork_tree.c)
6. Distinguishing Parent & Child process (Complete todos in fork_basic.c)
7. Demonstrate the usage of execlp (execlp.c)
8. Use execl to execute the exec_overlay program from exec_start (Complete todos in exec_start.c)
9. WAP to demonstrate the usage of execv, execve and execle
10. WAP to use fork & exec to have 2 processes executing different program (Complete todos in fork_execv.c)
11. WAP to demonstrate the usage of vfork
12. WAP to make the parent process wait for child process (Complete TODOS in wait.c)
13. Enhance the above program to check if the child process exiting normally or abnormally
14. Enhance the above program to create 2 child processes and wait for a particular child process
15. WAP to demonstrate the zombie process (Complete the todos in zombie.c)
16. WAP to demonstrate the orphan process and veriy if its adopted by Init(Complete the todos in orphan.c)

Signals

Exercises

Get into the directory by name 'Signals' & Complete the below assignments

1. WAP to handle SIGINT signal. (Complete TODOs in sig_int.c)
2. Modify the above program to display the number of times, the signal is received. The print should be done in main program.
3. Modify the above program to handle the SIGINT signal only once
4. Demonstrate the usage of SA_RESETHAND
5. WAP to Ignore the signal(SIGINT) for few iterations and then restore the behaviour
6. WAP to send the SIGUSR1 signal to the parent process (Try sig_usr.c)
7. WAP to mask a particular signal during the handler. For instance, mask SIGINT during the handling of SIGUSR1 (Complete todos in sig_mask.c)
8. WAP to handle the SIGALRM signal.
9. Modify the above program to send SIGALRM signal from the child process.
10. WAP to mask the particular signal for the process (complete todos in sig_procmask.c)
11. Enhance the above program to list the pending signal.
12. WAP to print the pid of the process which triggered the signal.
13. WAP to handle the SIGCHLD signal. Make sure to invoke the wait in the handler to clean up the child process (Modify zombie.c)
14. WAP to prevent the child from becoming zombie, even when the parent has not invoked the wait.
15. WAP to queue the signals.

Stage-1 of Project

Navigate to 'Stage1' directory under Project & complete the following:

1. Complete all the todos (1 to 5) in libsig.c
2. Complete todos in server.c to register the signal

Expected Output:

+ Compile the program using 'make'. This would generate server.exe and libsig.so
+ Execute the program using 'LD_LIBRARY_PATH=. server.exe'
+ Send the SIGINT signal by pressing 'Ctrl + C'. The handler should get invoked

IPC

Exercises

All the related exercises are placed under IPC folder

1. WAP to make Parent and Child communicate using the pipe (Complete all the todos in pipe.c)

2. WAP to enable the paging for the file reading. The parent process reads the file and passes the data to child process which executes the in-built paging utility ‘/bin/more’ to enable the paging for the file. (pipe2.c)

3. WAP to sort the text using the popen system call (refer popen.c)

4. WAP to sort the file listing by using the ls and sort command (complete todos in popen2.c)

5. WAP to make 2 processes communicate with FIFO using the system calls (complete todos in fifo_read.c and fifo_write.c)

6. WAP to make 2 processes communicate with FIFO using the library functions (Complete todos in fifo_client.c and fifo_server.c)

7. WAP to commmunicate 2 processes using the message queues (Complete TODOs in msg_recv.c and msg_send.c)

8. WAP to demonstrate the usage of shared memory (Complete todos in shared_mem.c)

9. WAP to make 2 processes communicate with each other using the shared memory and use binary semaphore to synchronize the access

Stage-2 of Project

Navigate to 'Stage2' directory under Project & complete the following:
1. Complete TODOs (1 to 6) in server.c
2. Complete TODOs (1 to 3) in client.c
Expected Output:
+ Compile the program using 'make'. This would generate server.exe and client.exe
+ Execute the program using 'LD_LIBRARY_PATH=. ./server.exe' in one shell
+ Open another shell & excecute ./client.exe
+ The client should display the Va, Vb, Vc as it receives from server

Stage-3 of Project

The objective for the stage-3 is to introduce the 'shared memory' as a communication mechanism between the processes 'register_update' & 'server'. Process 'register_update' periodically updates the EM registers in the shared memory, which in turn are being read by server.

Navigate to 'Stage3' directory under Project & complete the following:
1. Complete TODOs (1 to 7) in register_update.c
2. Complete TODOs (1 to 6) in libsem.c
3. Complete TODOs (7 to 13) in server.c
Expected Output:
+ Compile the program using 'make'. This would generate server.exe, client.exe and register_update.exe.
+ Execute the server program using 'LD_LIBRARY_PATH=. ./server.exe' in one shell
+ Open another shell & excecute ./client.exe.
+ The client should display the Va, Vb, Vc as it receives from server. It would display all the values as 0.
+ Open third shell and execute there register_update program using 'LD_LIBRARY_PATH=. ./register_update.exe'
+ The values of Va, Vb and Vc should change periodically at the client side

Threads

Exercises

All the relevant exercises are placed under the ‘Threads’ directory

1. Create a thread which displays a character infinitely (try thread.c)
2. Create 2 threads to print the character & make the main thread to wait on them (complete todos in thread_join.c)
3. Return the factorial of the number from the thread and display the same in main thread. (Complete the todos in thread_return.c)
4. WAP to create the detatched thread and try to get the return value from it. (Complete all the todos in thread_detach.c)
5. WAP to get the current stack size and set it to desired value.
6. WAP to get the current scheduling policy and sched param for thread and set the scheduling policy to SCHED_FIFO and priority to any value
7. WAP to cancel the thread. (Complete the todo 1 in thread_cancel.c)
8. Modify above program to disable the cancellation for the threads
9. Modify above program to set the cancellation type to deferred
10. Enhance the program to verify the return value of the thread on cancellation
11. WAP to register the clean up handler for the thread. (Complete todos in pthread_cancel_clean.c)
 + Try invoking pthread_exit before registering the handler
 + Try invoking pthread_exit after registering the handler
 + Try passing 0 to cleanup_pop function
12. WAP to demonstrate the usage of thread specific data area.(Complete todos in thread_specific.c). The program creates 5 threads and each of the threads are logging something into the specific log file. Use thread specific data area to store the file pointer for log file.
13. WAP to send the signal to the thread (Try thread_kill.c)
14. WAP to mask the signal.

Stage-4 of Project

The objective for the stage-4 is to have a multithreaded program where Em registers are updated periodically in a thread. The process 'register_update.c' has 3 threads as listed below:
 + update_registers threads which periodically generates the new value for the EM registers
 + update_shm: This is the main thread which periodically updates the shared memory with new values
 + calc_max: This thread monitors the max value of EM parameters and keeps updating the register set accordingly

 Navigate to 'Stage4' directory under Project & complete the following:
1. Complete TODOs (8 to 11) in register_update.c
Expected Output:
+ Compile the program using 'make'. This would generate server.exe, client.exe and register_update.exe.
+ Execute the server program using 'LD_LIBRARY_PATH=. ./server.exe' in one shell

+ Open another shell & excecute ./client.exe.
+ The client should display the Va, Vb, Vc as it receives from server. It would display all the values as 0.
+ Open third shell and execute there register_update program using 'LD_LIBRARY_PATH=. ./register_update.exe'

+ The values of Va, Vb, Vc, Vamax, Vbmax and Vcmax should change periodically at the client side

Thread Synchronization

Exercises

All the relevance exercises are placed under the directory name ‘Synchronization’

1. Modify cons_prod.c to protect global variable 'job_queue' and observe the behaviour
2. Modify cons_prod.c so that all threads keep on running forever
3. Modify thread_cond.c to use the conditional variables

Stage-5 of Project

The objective for the stage-5 is to Synchronize the access to the registers being concurrently accessed by the threads.

Navigate to 'Stage5' directory under Project & complete the following:
1. Complete TODOs (12 to 20) in register_update.c
Expected Output:
+ Compile the program using 'make'. This would generate server.exe, client.exe and register_update.exe.
+ Execute the server program using 'LD_LIBRARY_PATH=. ./server.exe' in one shell
+ Open another shell & excecute ./client.exe.
+ The client should display the Va, Vb, Vc as it receives from server. It would display all the values as 0.
+ Open third shell and execute there register_update program using 'LD_LIBRARY_PATH=. ./register_update.exe'
+ The values of Va, Vb, Vc, Vamax, Vbmax and Vcmax should change periodically at the client side

Socket Programming

Exercises

Get into the directory by name 'Networking' & Complete the below assignments:
1. Complete all the todos in server.c and client.c to communicate using the local sockets.
2. Complete all the todos in server1.c and client1.c to communicate using INET sockets.

Stage-6 of Project

The objective for the stage-6 is to make client & server communicate using socket
Navigate to 'Stage6' directory under Project & complete the following:
+ Complete all the todos in libsock.c

+ Complete TODOs 14 to 18 in server.c
+ Replace FIFO with socket in client.c

Output

+ Same as Stage-5, but communication between server and client would be using the socket